

Aim

To write a Python program for Bidirectional Associative Memory (BAM) with two pairs of vectors.

Objective

To study and understand the concept of Bidirectional Associative Memory in Artificial Neural Networks.

Important Theory for Practical Writing

Bidirectional Associative Memory (BAM) is a supervised learning neural network developed by Bart Kosko.

BAM is a type of recurrent neural network (RNN) used for associative memory. It stores pattern pairs and recalls one pattern when another related pattern is given.

Example:

- Face $\rightarrow$ Name
- Question $\rightarrow$ Answer
- Image $\rightarrow$ Text

This memory system works similarly to the human brain.

Why BAM is Required?

BAM is used:

- To store associated pattern pairs
 - To recover incomplete or noisy data
 - To perform bidirectional recall
 - To retrieve one pattern from another pattern
-

BAM Architecture

BAM contains:

1. X layer (Input Layer)
2. Y layer (Output Layer)
3. Weight matrix connecting both layers

Signals move in both directions:

- $X \rightarrow Y$
- $Y \rightarrow X$

This is why it is called Bidirectional Associative Memory.

Important Formula

The net input is calculated as:

$$net = \sum w_i x_i$$

Where:

- w_i = weights
 - x_i = input values
-

Activation Function

The activation function used is bipolar step function.

Rule:

- If value $\geq 0 \rightarrow$ Output = 1
- Else $\rightarrow$ Output = -1

Code

```
import numpy as np
```

```
# -----
```

```
# Bidirectional Associative Memory (BAM)
```

```
# -----
```

```
# Input patterns (Set A)
```

```
X = np.array([
```

```
    [1, 1, 1],
```

```
    [-1, -1, -1],
```

```
    [1, -1, 1],
```

```
    [-1, 1, -1]
```

```
])
```

```
# Target patterns (Set B)
```

```
Y = np.array([
```

```
    [1, 1, 1, 1, 1, 1],
```

```
    [-1, -1, -1, -1, -1, -1],
```

```

[1, -1, 1, -1, 1, -1],
[-1, 1, -1, 1, -1, 1]
])

# Initialize weight matrix
W = np.zeros((6, 3))

# Calculate BAM Weight Matrix
for i in range(len(X)):
    W += np.outer(Y[i], X[i])

print("Weight Matrix:")
print(W)

# Activation Function
def activation(x):
    return np.where(x >= 0, 1, -1)

# -----
# Testing for Input Patterns (Set A)
# -----

print("\nTesting for input patterns: Set A")

for i in range(len(X)):

    print("\nOutput of input pattern", i + 1)

# Recall target pattern
Y_test = activation(np.dot(W, X[i]))

print(Y_test.reshape(-1,1))

```

```
# -----
```

```
# Testing for Target Patterns (Set B)
```

```
# -----
```

```
print("\nTesting for target patterns: Set B")
```

```
for i in range(len(Y)):
```

```
    print("\nOutput of target pattern", i + 1)
```

```
        # Recall input pattern
```

```
        X_test = activation(np.dot(W.T, Y[i]))
```

```
        print(X_test.reshape(-1,1))
```